

On this page

Top Metrics to Monitor in Case Manager

In this section you will find which metrics are important to monitor in **Case Manager**.



Note that the following list is simply a reference to the most important metrics, and should not be considered as the single source of monitoring the healthiness of the system.

For a complete and comprehensive monitoring guide, please refer to the official [Case Manager documentation](#)

Please also refer to [Understand Available Metrics](#) for more information.



The following Case Manager standard application tags are available in every metric that references `Standard CM` application tags :

- `metricType`
- `appType`

Event Processing Metrics

New Event Throughput

Measures the event processing throughput when the event type is "new event", for the selected Case Manager Instance. An event is considered to be "new" if the identifier of that event has never been seen. Otherwise, it is seen as an update.

Metric	Description	Tags	Type
<code>cm.event.processing.late ncy</code>	The event processing throughput for "new events"	<code>eventType</code> - Standard CM application tags	Counter

What to Monitor

This metric is useful to ensure that the rate at which events are being processed is as expected. If this rate is lower than what Pulse is processing, then it could mean that there is a problem in the system, and there will be backpressure on RabbitMQ.

New Event Latency

Measures the event processing latencies per percentile (50th, 95th, 99th, and 99.9th) when the event type is "new event", for the selected Case Manager instance.

Metric	Description	Tags	Type
<code>cm.event.processing.l atency</code>	The event processing latency for "new events" at various percentiles.	<code>eventType</code> - Standard CM application tags	Timer

What to Monitor^â

This metric is useful together with the previous throughput metric. As it reports the latency of the new events processed, if the throughput starts to decrease, we can use this metric as a complement to understand what can be contributing to that. For example, the system might be reaching memory exhaustion (see JVM metrics), or it's taking too long to run all the automations configured to trigger on event arrival.

Event Processing Queue Occupancy^â

Measures the percentage of the event processing queue being used, for the selected Case Manager instance.

Metric	Description	Tags	Type
<code>cm.event.processing.queue.ratio</code>	The percentage of the event processing queue being used.	Standard CM application tags	Gauge

What to Monitor^â

This metric shows whether events are being queued in Case Manager's internal processing queue or not.

Event Queue Deadletter^â

The Event Queue deadletter is the process by which events that could not be processed successfully are written to local storage to be re-attempted later on by an automatic process. The retry outcomes are also measured.

Metric	Description	Tags	Type
<code>cm.event.queues.dump.storage.dump.entries.failed.counter</code>	The total number of failed event queue entries dumped to storage.	Standard CM application tags	Counter
<code>cm.event.queues.dump.storage.dump.entries.retry.counter</code>	The total number of retried event queue entries dumped to storage.	Standard CM application tags	Counter

What to Monitor^â

Events are not expected to be spilled to disk, and hence, this metric should report zero. Otherwise, this indicates that some events are failing to be ingested from the upstream queue and processed. This could happen, for example, if there is an automation or custom extension being invoked on event arrival that failed exceptionally.

Event Storage Deadletter^â

The Event Storage deadletter is the process by which events were processed but could not be persisted successfully. These events are written to local storage to be re-attempted later on by an automatic process. The retry outcomes are also measured.

Metric	Description	Tags	Type
<code>cm.event.storage.dump.storage.dump.entries.failed.counter</code>	The total number of failed event storage entries dumped to storage.	Standard CM application tags	Counter
<code>cm.event.storage.dump.storage.dump.entries.retry.counter</code>	The total number of retried event storage entries dumped to storage.	Standard CM application tags	Counter

What to Monitor^â

Events are not expected to be spilled to disk, and hence, this metric should report zero. Otherwise, this indicates that some events are failing to be persisted to the database. This could happen, for example, if there is a connectivity issue with the database, or if the operation is failing exceptionally.

Automation Rules Metrics^â

Event Automations Throughput^â

Measures the throughput of the automation rules execution, for a given automation rule identified by the tag `ruleName` .

Metric	Description	Tags	Type
<code>cm.automation</code>	The throughput of automation rule execution.	<code>ruleName</code> - Standard CM application tags	Counter

What to Monitor^â

This metric is useful to ensure that the rate at which specific automations are being triggered is as expected. If you have a flow that depends on specific rules that trigger on event updates and such automations are not triggering, then it can indicate a failure in some part of the integration.

Event Automations Latency^â

Measures the latency per percentile (50th, 95th, 99th, and 99.9th) of the automation rules execution, for a given automation rule identified by the tag `ruleName` .

Metric	Description	Tags	Type
<code>cm.automation</code>	The latency of automation rule execution at various percentiles.	<code>ruleName</code> - Standard CM application tags	Timer

What to Monitor^â

This metric is useful together with the previous throughput metric. As it reports the latency of the automations being executed, if the throughput starts to decrease, we can use this metric as a complement to understand what can be contributing to that. For example, specific actions of certain automations could be inflicting some overhead in the system.

Event Automations Errors^â

Measures the errors observed during the execution of automation rules, for individual rules. The rule is identified by the tag `ruleName` .

Metric	Description	Tags	Type
<code>cm.automation.error</code>	The rate of errors returned by the automation execution.	<code>ruleName</code> - Standard CM application tags	Meter

What to Monitor^â

This metric provides a simple way to understand if the automations execution is failing during its execution. When there are no errors, the value of this counter is zero. If this metric reports a different value, then you must look into the log files for additional error details.

Integration Metrics¹

Pulse Requests Errors¹

Measures the errors returned by the Pulse Server API, for the endpoint described in the path tag. Case Manager engages with Pulse for several purposes, such as propagating fraud feedbacks, updating list items, validating user roles and permissions, etc. Each operation is done in a different endpoint.

Metric	Description	Tags	Type
<code>cm.request.pulse.api.error</code>	The rate of errors returned by the Pulse endpoint.	<code>path</code> - Standard CM application tags	Meter

What to Monitor¹

This metric provides a way to monitor the integration between Case Manager and Pulse, where errors are not expected to occur. If the error count over time is greater than zero, then that means that either there is a network issue preventing Case Manager from contacting Pulse, or Pulse is facing problems. In any case, if there are errors it means that some operations have failed and may need to be retried (e.g., listing entities).

Infrastructure Metrics¹

Database Connection Pool¹

Each instance of Case Manager has a lazy connection pool to a database. One of the most important to monitor is the one used by the Apps Service. This service runs in Pulse Server and is responsible for handling all metadata management. The metrics for the DB pool are available under the `cm.db.connection.pool` prefix.

The metrics of the DB pool are available under the `cm.db.connection.pool` prefix.

Metric	Description	Tags	Type
<code>cm.db.connection.pool.size</code>	The current size of the connection pool (if pool is lazy, the value can grow over time).	Standard CM application tags	Gauge
<code>cm.db.connection.pool.free</code>	The number of free database connections.	Standard CM application tags	Gauge
<code>cm.db.connection.pool.used</code>	The number of used database connections.	Standard CM application tags	Gauge
<code>cm.db.connection.pool.waiting</code>	The number of threads waiting for a database connection.	Standard CM application tags	Gauge
<code>cm.db.connection.pool.usage</code>	The current usage of the database connection pool (used/size in percent).	Standard CM application tags	Gauge

What to Monitor¹

These metrics detect underlying issues with connecting to the DB, or potential problems interacting with it. For example, if connections are not being released because operations are blocked, or if the database is becoming a bottleneck. If the pool usage is always at close to 100%, then there's likely an issue.

JVM Metrics

Each Pulse Server and Pulse Application instance lives in their own JVM process. Therefore, JVM metrics for each of these processes are available. JVM metrics are available as `letmeknow.jvm`. You'll find many different metrics under this namespace from GC to Memory.

What to Monitor

Look into at least GC and Memory metrics. These give you an idea of the healthiness of the process.

Other Monitoring Metrics



The following metrics are still being improved and they might change in the near future.

Metric	Description	Tags	Type
<code>hsbc.afis.responseMeter.httpStatusCode</code>	Operational results (using the count value).	N/A	Counter
<code>hsbc.afis.retry</code>	Number of requests retried (using the count value).	N/A	Counter
<code>hsbc.afis.fallback</code>	Measures the number of events dumped to disk (using the count value).	N/A	Counter
<code>hsbc.afis.requestLatencyHistogram</code>	Requests latency (using the available percentile values).	N/A	Counter